
Interactions Between Pruning, Quantization, and Batch Size in MLPs

Hema
IIT Madras Zanzibar
zda24b025@iitmz.ac.in

Abstract

Neural networks are usually larger than they need to be, which makes them slow to run and heavy to store. Compression methods such as pruning and quantization shrink them, but they are typically studied in isolation. This work studies how these methods behave and how they interact when applied to Multi-Layer Perceptrons (MLPs). In this stage I implement each compression method on its own: pruning, quantization, coreset selection and PCA. I have compared three classical machine learning methods against an MLP on two MedMNIST datasets PneumoniaMNIST and BloodMNIST. The MLP matches or beats the classical models. It can be pruned to 90% sparsity and quantized to 3 bits with negligible accuracy loss after fine-tuning and PCA sharply reduces input dimensionality and in one case improving accuracy. These baselines set up the interaction study to follow which asks whether the order of pruning and quantization matters and whether the best batch size shifts after compression and whether combining methods beats any one alone.

1 Introduction

Neural networks are often larger than they need to be which makes them slow to run and heavy to store. Compression methods like pruning (removing weights) and quantization (storing weights with fewer bits) shrink them but most work studies these methods one at a time. The goal of this project is to study how these methods behave and eventually how they interact when applied to MLPs. I implemented each compression method on its own and compare classical machine learning against the MLP setting up the baselines for the later interaction study. The research questions driving the project are: (1) does the order of pruning and quantization matter; (2) does the best batch size shift once a model is compressed; and (3) does combining methods beat using any one alone?

2 Related Work

Pruning has a long history beginning with second-order methods such as Optimal Brain Damage (1), which removes weights using curvature information. Modern magnitude pruning was popularised by Han et al. (2), who showed that removing low-magnitude weights and retraining preserves accuracy. The Deep Compression pipeline (3) combined pruning with quantization and Huffman coding to achieve large size reductions. The Lottery Ticket Hypothesis (4) showed that sparse trainable subnetworks exist inside dense ones. For batch size, Keskar et al. (5) showed that large-batch training tends to converge to sharp minima that generalise worse, and Smith et al. (6) showed batch size and learning rate are partly interchangeable. The datasets used here come from the MedMNIST benchmark (7), which provides standardised lightweight medical image classification tasks. Most of these works study a single compression axis; this project instead examines how several interact.

3 Methodology

Classical models: Three classical methods serve as comparison points: Logistic Regression, Random Forest, and an RBF-kernel SVM each taking the flattened pixel vectors directly. These indicate whether a neural network is worth using.

MLP: The neural model is a 3-layer MLP (input $\rightarrow$ 512 $\rightarrow$ 256 $\rightarrow$ output) with ReLU activations and dropout, trained with Adam and cross-entropy loss. Validation accuracy is tracked each epoch. The best version is kept and training stops early when it stops improving. This MLP is the model all compression methods are applied to.

Pruning: I used magnitude pruning, rank all weights and set the smallest fraction to zero, sweeping sparsity from 10% to 90%. Accuracy is measured immediately (raw damage) and after a short fine-tuning step that lets the surviving weights re-adjust and recover.

Quantization: I use simulated post-training quantization, each weight is rounded to one of 2^b evenly-spaced levels between the smallest and largest weight, sweeping the bit-width b from 8 down to 2.

Coreset selection: The MLP is trained on random subsets of the data (25%, 50%, 100%) to test how much data is needed.

Dimensionality reduction: PCA is applied to the input before the MLP sweeping the number of retained dimensions (16, 32, 64, 128).

4 Experiments

All experiments use a fixed seed (42) for reproducibility. The MLP uses Adam with weight decay 10^{-4} , batch size 128, dropout 0.5 (Pneumonia) or 0.3 (Blood) and learning rate 3×10^{-4} (Pneumonia) or 10^{-3} (Blood), with early stopping on validation accuracy. The MLP baseline is reported as mean $\pm$ standard deviation over 3 seeds (42, 43, 44); other results are single-run. Pneumonia uses heavier regularisation because its test split is drawn from a different part of the source data than its training split, creating a natural accuracy ceiling. Pruning fine-tuning uses 10 epochs at learning rate 5×10^{-4} . Experiments were run in Google Colab. Evaluation metrics are test accuracy and macro-F1; macro-F1 is reported because PneumoniaMNIST is imbalanced. Datasets are summarised in Table 1.

Table 1: Dataset summary. Two MedMNIST datasets of differing modality and class count.

	PneumoniaMNIST	BloodMNIST
Type	Grayscale	RGB
Classes	2	8
MLP input size	784	2352
Train / Val / Test	4708 / 524 / 624	11959 / 1712 / 3421
Balance	Imbalanced	Mostly balanced

5 Results

Table 2: Classical methods vs MLP (test accuracy / macro-F1). MLP accuracy is mean $\pm$ std over 3 seeds. The MLP wins clearly on Blood.

Model	Pneu Acc	Pneu F1	Blood Acc	Blood F1
Logistic Regression	0.832	0.799	0.786	0.763
Random Forest	0.853	0.829	0.838	0.810
RBF SVM	0.857	0.832	0.853	0.832
MLP	0.858 ± 0.002	0.833	0.867 ± 0.002	0.842

On Pneumonia all models land around 85% and none clearly wins, due to the dataset’s train/test source difference. On Blood the MLP wins, pulling ahead on cleaner, harder multi-class data.

Table 3: Pruning: test accuracy before and after fine-tuning. Even at 90% sparsity, accuracy recovers to baseline or better.

Sparsity	Pneumonia		Blood	
	Before	After	Before	After
50%	0.849	0.867	0.865	0.873
70%	0.841	0.861	0.863	0.871
80%	0.832	0.870	0.779	0.870
90%	0.838	0.846	0.478	0.874

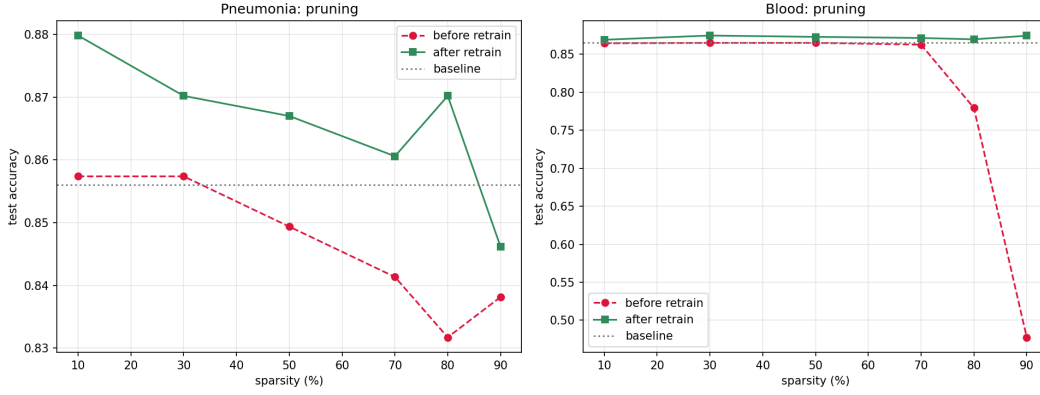


Figure 1: Enter Caption

Blood drops sharply before retraining (0.478 at 90%) but recovers fully (0.874), showing the network is roughly 90% redundant.

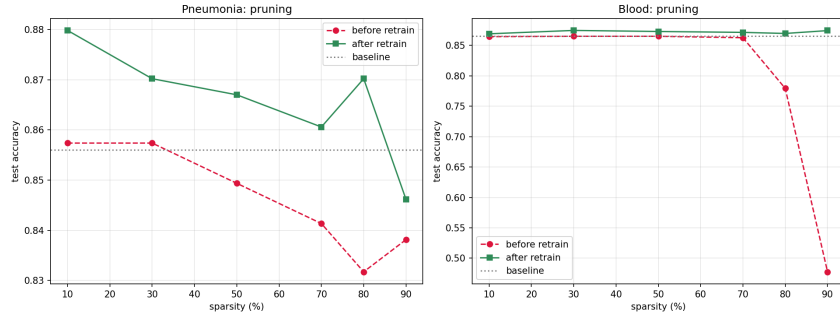


Figure 2: Pruning accuracy before (red) and after (green) fine-tuning, versus sparsity. The sharp drop at high sparsity is fully recovered after a short retraining step, showing the networks are highly redundant.

Coreset shows 50% of the data reaches most of the accuracy. PCA reduces 784 inputs to 128 dimensions while *improving* Pneumonia accuracy to 0.883, because PCA removes noise; Blood needs more dimensions since RGB images carry more information.

6 Discussion

Each compression method works well on its own. Pruning to 90% sparsity and quantizing to 3 bits both keep accuracy near baseline, and PCA shrinks the input dramatically. A key limitation is that the MLP discards the spatial structure of images, so its accuracy sits below a CNN's; the focus here is the compression behaviour, not absolute accuracy. The natural next step is the interaction study: whether prune-then-quantize differs from quantize-then-prune, whether the best batch size changes after compression, and whether a combined strategy beats any single method.

Table 4: Quantization: test accuracy vs bit-width. Lossless to 3 bits, breaks at 2.

Bits	Pneumonia	Blood
32 (full)	0.856	0.865
8	0.856	0.865
4	0.859	0.862
3	0.861	0.865
2	0.848	0.718

7 Conclusion

This stage established classical and MLP baselines on two MedMNIST datasets and showed that each compression method, pruning, quantization, coresets selection, and PCA, can substantially shrink the model or input with little accuracy loss. These results form the foundation for studying how the methods interact.

References

- [1] Y. LeCun, J. Denker, S. Solla (1990) Optimal Brain Damage. *Advances in Neural Information Processing Systems 2*.
- [2] S. Han, J. Pool, J. Tran, W. Dally (2015) Learning both Weights and Connections for Efficient Neural Networks. *NeurIPS*. <https://arxiv.org/abs/1506.02626>
- [3] S. Han, H. Mao, W. Dally (2016) Deep Compression. *ICLR*. <https://arxiv.org/abs/1510.00149>
- [4] J. Frankle, M. Carbin (2019) The Lottery Ticket Hypothesis. *ICLR*. <https://arxiv.org/abs/1803.03635>
- [5] N. Keskar et al. (2017) On Large-Batch Training for Deep Learning. *ICLR*. <https://arxiv.org/abs/1609.04836>
- [6] S. Smith et al. (2018) Don't Decay the Learning Rate, Increase the Batch Size. *ICLR*. <https://arxiv.org/abs/1711.00489>
- [7] J. Yang et al. (2023) MedMNIST v2. *Scientific Data*. <https://www.nature.com/articles/s41597-022-01721-8>